AD Aflevering 4

kvb125, fps618

Mar 2024

1 Rød-sorte søgetræer

For at løse denne opgave, skal vi forstå de grundlæggende regler for rød-sort træer som beskrevet i CLRS. Reglerne for rød-sort træer er:

- Hver knude er enten rød eller sort.
-Roden er altid sort.
-Alle blade (NIL knuder) er sorte.
-Hvis en knude er rød, så er begge dens børn sorte.
-For enhver knude gælder, at alle simple stier fra knuden til efterkommernes blade indeholder det samme antal sorte knuder.

For at finde ud af, hvilke knuder der er mulige på de uspecificerede positioner, skal vi overveje værdierne og farverne på de omkringliggende knuder, samtidig med at vi sikrer, at rød-sort træets regler overholdes.

Her er en gennemgang af hvert punkt:

- En rød knude med nøglen 28 kan ikke placeres umiddelbart under den sorte 19-knude, da dette vil kræve, at 28 har en sort forælder, og 19 allerede har et rødt barn, 24. Den kan heller ikke placeres under den røde 47, da det ville krænke reglen om, at røde knuder ikke kan have røde børn.

- En rød knude med nøglen 44 kan placeres som barn til den sorte knude 42, men ikke som barn til den røde 47, da det ville krænke reglen om røde knuder ikke kan have røde børn.

- En rød knude med nøglen 58 kan ikke placeres under den røde 56-knude, da dette ville krænke reglen om, at røde knuder ikke kan have røde børn.

- En sort knude med nøglen 25 kan ikke placeres under den sorte 19-knude, da dette ville krænke binær søgetræegenskaben (alle nøgler i venstre subtree er mindre end forældrenøglen).

- En sort knude med nøglen 30 kunne teoretisk placeres under den sorte 19-knude, hvis vi antager, at børnene til denne nye knude ville blive struktureret korrekt til at bevare træets egenskaber.

- En sort knude med nøglen 66 kan placeres som et barn af den sorte 56-knude for at bevare både den binære søgeegenskab og rød-sort træegenskaberne.

Derfor kan vi konkludere:

- **Mulighed 2** (en rød knude med nøglen 44) og **mulighed 6** (en sort knude med nøglen 66) er gyldige positioner for de manglende knuder, da de opretholder træets invarianter.

- **Mulighed** 5 (en sort knude med nøglen 30) kunne også være en mulighed, afhængigt af de specifikke børn og træstrukturen omkring denne knude.

De øvrige muligheder (1, 3, og 4) overholder ikke rød-sort træets invarianter, når de placeres ved nogen af de uspecificerede knuder.

2 Indsættelse i rød-sorte søgetræer

For at bestemme, hvad der sker, når vi indsætter nøglen 9 i det rød-sorte binære søgetræ, kan vi følge trinene i RB-INSERT algoritmen:

1. Da træet ikke er tomt, begynder vi ved roden og søger for at finde det rigtige sted til nøglen 9.

2. Vi sammenligner 9 med nøglen i roden (10). Da $9 < 10$, bevæger vi os til venstre barn af 10, som er 3.

3. Nu sammenligner vi 9 med 3. Da $9 > 3$, bevæger vi os til højre barn af 3, som er 8.

4. Da 8 ikke har et højre barn (vi antager at bladene eller T.nil ikke vises), har vi fundet stedet, hvor 9 skal indsættes.

5. Ifølge algoritmen, vil 9 blive z's forælder's højre barn (da $9 > 8$), og den nye knude vil blive farvet rød.

Derfor vil det korrekte svar på opgaven være:

9 indsættes som højre barn til knude 8 og farves rød.

Vi behøver ikke at udføre RB-INSERT-FIXUP i dette tilfælde endnu, da vi kun skal identificere den umiddelbare ændring efter indsættelse, før nogen rebalancing er sket.

3 Køretid for rød-sorte søgetræer

For at vurdere disse udsagn, kigger vi på de egenskaber og operationer for et rød-sort træ beskrevet i CLRS kapitel 13.

Den samlede tid for operationerne er $\Omega(N)$: Dette udsagn er sandt. Da der udføres i alt N operationer, og hver operation (indsættelse, sletning, eller søgning) i værste fald tager konstant tid (for eksempel, en mislykket søgning kan stadig finde sted i $O(1)$ hvis det første element, der tjekkes, er det ønskede element), vil den samlede tid være mindst proportional med antallet af operationer, hvilket er $\Omega(N)$.

Den samlede tid for operationerne er $O(N \lg N)$: Dette udsagn kan være sandt eller falsk afhængig af hvordan operationerne fordeler sig. Hvis vi antager, at alle N operationer er indsatser eller sletninger, så vil den samlede tid være $O(N \lg N)$, fordi hver af disse operationer tager $O(\lg N)$ tid i et træ med N elementer. Men hvis der for eksempel kun er én indsættelse og resten er søgeoperationer, vil den samlede tid være tættere på $O(N)$ end $O(N \lg N)$. Udsagnet er dog ikke nødvendigvis falsk, da $O(N \lg N)$ ikke er en præcis øvre grænse, men en grænse for det værst tænkelige tilfælde af en blandet sekvens af operationer.

Den samlede tid for operationerne er $O\left(n_3 + (n_1 + n_2) \lg N\right)$: Dette udsagn er sandt. Søgeoperationer (n3) tager $O(\lg N)$ tid hver, men når vi udfører mange af dem, er deres samlede tid $O(n_3)$. For indsættelser og sletninger (n1 og n2), tager hver operation $O(\lg N)$ tid, og derfor er den samlede tid for disse $O((n_1 + n_2) \lg N)$. Så det samlede tidskompleksitet for alle operationerne vil være $O\left(n_3 + (n_1 + n_2) \lg N\right)$.

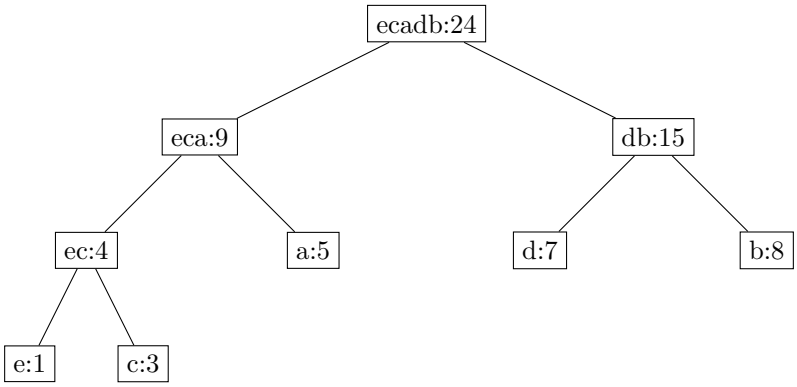
Alle blade i T er i dybde mindst $\lfloor \lg (n_3 + 1) \rfloor$: Dette udsagn er ikke nødvendigvis sandt. Antallet af søgeoperationer (n3) har ingen direkte indflydelse på dybden af bladene i et rød-sort træ. Dybden af bladene afhænger af antallet af indsættelser og sletninger, der ændrer træets struktur. Derudover garanterer egenskaberne for rød-sort træ, at de længste stier fra roden til et blad er højst dobbelt så lange som de korteste stier, hvilket holder træet tilnærmelsesvis balanceret. Men der er ingen direkte forbindelse mellem antallet af søgninger og minimumsdybden af bladene.

Alle sorte knuder i T har en afstand til roden, der er et lige tal: Dette udsagn er falsk. I et rød-sort træ er der ingen sådan regel, der dikterer, at alle sorte knuder skal have en lige afstand til roden. Rød-sort træer har egenskaber, der sikrer, at ingen rød knude har et rødt barn, og at alle stier fra roden til bladene har det samme antal sorte knuder, men det kræver ikke, at disse afstande er lige tal.

Derfor er udsagn 1 og 3 korrekte. Udsagn 5 og 6 er ikke korrekte, og udsagn 2 er betinget korrekt, men ikke strengt taget nødvendigt sandt for alle sekvenser af operationer.

4 Huffman koder

Vi betragter algoritmen til konstruktion af Huffman koder som beskrevet i CLRS sektion 15.3 for alfabetet $C = \{a, b, c, d, e\}$ med frekvenser $f_a = 5, f_b = 8, f_c = 3, f_d = 7, f_e = 1$. Her er det beregnede Huffman træ med angivelse af kombinerede bogstaver ved hvert knudepunkt:



I træet repræsenterer hver knude en kombination af karakterer fra alfabetet med deres samlede frekvens. Træets konstruktion følger Huffman-algoritmen, hvor de to tegn med de laveste frekvenser kombineres først, og denne proces gentages, indtil der opnås et enkelt træ, der repræsenterer den mest effektive kode. De endelige blade i træet repræsenterer de oprindelige karakterer og deres frekvenser, mens de indre knuder repræsenterer kombinationer af disse karakterer.

5 Dynamisk programmering

a)

Vi skal argumentere for at $t_{i,j} \geq 2^{j-i+1} - 1$ ved at anvende induktion i $\ell = j - i$.

Basis for induktionen ($\ell = 0$, hvilket betyder $i = j$):

- For $i = j$, ifølge definitionen er $t_{i,j} = 1$.
- Da $2^{j-i+1} - 1 = 2^{0+1} - 1 = 2 - 1 = 1$, så er $t_{i,j} = 1 = 2^{j-i+1} - 1$.
- Derfor gælder påstanden for $\ell = 0$.

Induktionsantagelse: Antag at $t_{i,j} \geq 2^{j-i+1} - 1$ er sandt for alle i, j således at $j - i = \ell$.
Induktionsskridt ($\ell + 1$): Vi skal vise, at $t_{i,j} \geq 2^{j-i+1} - 1$ også gælder for $\ell + 1$, det vil sige når $j - i = \ell + 1$.
Fra definitionen af $t_{i,j}$ for $i < j$, har vi:

$$t_{i,j} = 1 + \sum_{k=i}^{j-1} t_{i,k} + t_{k+1,j}$$

Ifølge induktionsantagelsen antager vi, at for ethvert interval mindre end $\ell + 1$, er $t_{i,k} \geq 2^{k-i+1} - 1$ og $t_{k+1,j} \geq 2^{j-k} - 1$.

Det mindste interval vi kan have er når $k = i$ eller $k = j - 1$, hvilket vil sige, at både $t_{i,k}$ og $t_{k+1,j}$ dækker mindst et interval af længden 1 (eller $\ell = 0$, som allerede er bevist til at være mindst 1).

For hvert trin k fra i til $j - 1$ opdeles intervallet $[i, j]$ i to dele: $[i, k]$ og $[k + 1, j]$, hver med mindst en størrelse på 1, som ved induktionsantagelsen hver bidrager med mindst $2^{k-i+1} - 1 + 2^{j-k} - 1$ til summen.

Summen over alle mulige k fordobler antallet af termer (hver opsplitning fører til to nye intervaller), så for hvert niveau af opsplitning fordobles antallet af termer i summen. Da vi starter med $\ell + 1$ som mindste intervalstørrelse, vil det samlede antal termer, vi tilføjer, være mindst $2^{\ell+1}$.

Så,

$$t_{i,j} = 1 + \sum_{k=i}^{j-1}(t_{i,k} + t_{k+1,j}) \geq 1 + \sum_{k=i}^{j-1}(2^{k-i+1} - 1 + 2^{j-k} - 1) \geq 1 + (j-i) \cdot 2^{j-i} > 2^{j-i+1} - 1$$

Dette fuldfører induktionsskridtet og viser, at $t_{i,j} \geq 2^{j-i+1} - 1$ for alle $i \leq j$.

b)

For at beregne $m_{i,j}$ for alle $1 \leq i \leq j \leq n$ effektivt ved hjælp af dynamisk programmering, anvender vi følgende algoritme, som gemmer de beregnede værdier for at undgå gentagne beregninger.

Algorithm 1 Beregn $m_{i,j}$ ved hjælp af dynamisk programmering

```
Initialiser en tabel M[n+1][n+1] til at holde m værdier, sat til −1 eller en anden markør for "ikke beregnet"
for i = n to 1 do
  for j = i to n do
    if i == j then
      M[i][j] = 1
    else
      for k = i to j − 1 do
        val = M[i][k] + M[k+1][j] + x[i−1] · x[k] · x[j]
        if val > M[i][j] then
          M[i][j] = val
        end if
      end for
    end if
  end for
end for
Returner M[1][n]
```

Køretidsanalysen viser, at algoritmen har en køretid på $O(n^3)$, da den udfører et tredobbelt for-loop for at beregne hver $m_{i,j}$. Pladsanalysen viser en pladskompleksitet på $O(n^2)$ på grund af brugen af en $n \times n$ tabel.

6 Grådige algoritmer

a)

For at argumentere for, at der i en optimal løsning I^* altid vil være et lige antal elementer i mængden $I^*_- = \{i \in I^* \mid x_i < 0\}$, overvej følgende punkter:

1. **Produktet af et lige antal negative tal er positivt:** Multiplikation af et lige antal negative tal resulterer altid i et positivt produkt, da hvert par af negative tal giver et positivt resultat.
2. **Produktet af et ulige antal negative tal er negativt:** Når antallet af negative tal, der multipliceres, er ulige, vil det endelige produkt være negativt.
3. **Maksimering af produktet:** For at maksimere produktet af en delmængde af tal er det optimalt at inkludere et lige antal negative tal, hvis nogen, for at sikre, at deres produkt bidrager positivt til det samlede produkt.
4. **Optimal løsning og negative tal:** Derfor, hvis der findes negative tal i den optimale delmængde I^*, skal antallet af disse negative tal være lige for at opnå den højeste mulige produktværdi.

Konklusionen er, at for enhver optimal løsning I^*, vil $\left|I^*_-\right|$, antallet af negative tal i I^*, være deleligt med 2 for at maksimere produktet $\prod_{i \in I^*} x_i$.

b)

Vi foreslår følgende effektive algoritme for at finde den optimale løsning I^* til MaxProduct-problemet og argumenterer for dens korrekthed og køretid.

Algorithm 2 Find den optimale løsning til MaxProduct-problemet

```
Sortér inputlisten x_1, ..., x_n i ikke-aftagende rækkefølge
if listen indeholder positive tal then
    Fjern alle 0'er
end if
Opdel tallene i positive og negative tal
if antallet af negative tal er ulige then
    Fjern det negative tal tætst på 0 (det største negative tal)
end if
Kombiner alle resterende tal for at finde det endelige produkt
```

Korrekthed:

- Inkludering af *alle positive tal* i I^* øger altid det samlede produkt, da multiplikation med et tal større end 1 øger produktet.
- Ved at *inkludere et lige antal negative tal*, sikrer vi et positivt produkt. Udelukkelse af det største negative tal (nærmest 0) i tilfælde af et ulige antal negative tal maksimerer det samlede produkt.
- *Nulværdier* fjernes for at undgå et samlet produkt på 0, hvilket ikke er optimalt, medmindre alle tal er 0 eller negative.

Køretid:

- Sortering af listen kræver $O(n \log n)$ tid.
- Gennemgang af listen for at opdele tal og fjerne 0'er kræver $O(n)$ tid.
- Samlet set er køretiden domineret af sorteringstrinnet, hvilket giver en køretid på $O(n \log n)$.

Denne tilgang sikrer, at vi effektivt finder den optimale løsning I^* ved korrekt at kombinere de sæt af positive og negative tal, der maksimerer det samlede produkt.